

Sistemas Distribuidos y Paralelos - Resumen de Promoción

Contents

1	Sistemas Paralelos	5
1.1	Concurrencia vs. Paralelismo	5
1.2	Sistema paralelo	5
1.3	¿Como mejorar el rendimiento?	5
1.4	Tipos de paralelismo	6
1.4.1	Paralelismo implícito	6
1.4.2	Paralelismo explícito	6
1.4.3	Comportamiento de las aplicaciones	6
1.4.4	Sistemas Operativos	7
1.4.5	Lenguajes de Programación	7
1.5	Modelos de programación paralela	7
1.6	Características del desarrollo de aplicaciones paralelas	7
1.7	High Performance Computing (HPC)	8
2	Memoria	9
2.1	Limitaciones del Sistema de memoria	9
2.2	Memoria Cache	9
2.2.1	Principio de localidad	9
2.2.2	Estados y Niveles de la memoria Cache	9
2.2.3	Problemas de cache	10
2.2.4	Optimización utilizando principio de localidad	10
2.2.5	Minimizando el espacio de memoria	10
3	Clasificación de arquitecturas paralelas	12
3.1	Por su organización lógica	12
3.1.1	Mecanismos de control (Taxonomía de Flynn)	12
3.1.2	Modelos de comunicación	13
3.2	Por su organización física	13
3.2.1	Espacio de direcciones	13
3.2.2	Red de interconexión	14
3.2.3	Granularidad	14
4	Evolución de las arquitecturas paralelas	14
4.1	Paralelismo en procesadores	14
5	Diseño de algoritmos paralelos (estrategias de Ian Foster)	16
5.1	Descomposición o Particionado	16
5.1.1	Descomposición recursiva	16
5.1.2	Descomposición basada en los datos	16
5.1.3	Descomposición exploratoria	17
5.1.4	Descomposición especulativa	17
5.1.5	Descomposición híbrida	17
5.2	Comunicación	17
5.3	Aglomeración	18
5.4	Mapeo	18
5.4.1	Mapeo estático	18
5.4.2	Mapeo dinámico	19
6	Modelo de programación sobre memoria compartida	20
6.1	No confundir	20
6.2	Pthreads	20
6.2.1	Programa básico	20
6.2.2	Sincronización	21
6.2.3	Afinidad	21
6.3	OpenMP	21

6.3.1	Estructura de control paralela	21
6.3.2	Trabajo compartido	21
6.3.3	Entorno de datos	22
6.3.4	Sincronización	22
6.3.5	Afinidad	22
7	Modelo de programación sobre memoria distribuida	23
7.1	Message Passing Interface (MPI)	23
7.1.1	Funcionamiento general	23
7.1.2	Estructura de programa	23
7.1.3	Comunicación	24
8	Modelo de programación híbrido	24
8.1	Modularidad	24
9	Métricas	25
9.1	Speedup	25
9.2	Eficiencia	26
9.3	Overhead	26
9.4	Balance de carga	26
9.5	Escalabilidad en los sistemas paralelos	26
9.5.1	Ley de Amdahl	26
9.5.2	Ley de Gustafson	27
9.5.3	Escalabilidad	27
9.5.4	Modelo de isoeficiencia	28
9.6	Métricas en sistemas con arquitecturas heterogéneas	28
9.7	Ejecución paramétrica	29
10	Paradigmas de programación paralela	30
10.1	Paralelismo de datos - SIMD/SPMD	30
10.2	Divide y vencerás	30
10.3	Pipelines	30
10.4	Algoritmos sistólicos	31
10.5	Master-Worker	31
10.6	Task pools	31
10.7	Modelos Híbridos	31
11	Resumen de Programación CUDA	33
11.1	¿Por qué usar GPUs?	33
11.2	Arquitectura de una GPU Nvidia	33
11.3	Modelo de Programación CUDA	33
11.4	Kernel y ejecución	33
11.5	Identificación de hilos	34
11.6	Limitaciones	34
11.7	Tensor Cores	34
11.8	Métricas comunes	34
11.9	Depuración	34
12	Jerarquía de memoria de GPUs	34
12.1	CUDA Capability	34
12.2	Características generales de la jerarquía de memoria Nvidia	34
12.3	Descripción de las distintas memorias de la jerarquía NVidia	35
12.3.1	Memoria global	35
12.3.2	Memoria de constantes	35
12.3.3	Memoria de texturas	36
12.3.4	Memoria compartida	36
12.3.5	Registros	36

12.3.6 Memoria local	36
13 Optimizaciones CUDA	37
13.1 Coalescense: Organización de los accesos	37
13.2 Prefetching: Técnicas de carga anticipada de datos	37
13.3 Divergence: Relacionado a la ejecución de los threads	37
13.4 Mezcla y granularidad: técnica de optimización en GPU	37
13.5 Occupancy: asignación de recursos en un SM	38
14 CUDA Streams	38
15 MultiGPUs y modelos híbridos	38

1 Sistemas Paralelos

1.1 Concurrencia vs. Paralelismo

La **concurrencia** es un **concepto de software** que permite la simultaneidad en la ejecución de múltiples actividades. Cada tarea es ejecutada por múltiples procesos los cuales **pueden ser independientes, competir o cooperar**.

Por otro lado, el **paralelismo** es un **concepto de hardware** (que involucra tanto software como hardware). Básicamente, el paralelismo es **concurrencia que requiere de hardware adicional**. Los procesos se ejecutan en mas de una unidad de procesamiento, es por ello que se trata de un concepto de hardware. Su **objetivo** es mejorar el rendimiento de una aplicación, ya sea en tiempo o energía.

1.2 Sistema paralelo

Un sistema paralelo es la combinación entre un **algoritmo paralelo** y la **arquitectura paralela** sobre la que se implementa.

- **Software paralelo (algoritmo paralelo):** varios procesos o hilos que cooperan para la resolución de un problema, con el objetivo de lograr **mayor rendimiento** que un algoritmo secuencial.
- **Hardware paralelo (arquitectura paralela):** arquitectura con múltiples unidades de procesamiento que permiten el procesamiento paralelo del software paralelo.

Es importante este concepto, Paralelismo = Software + Hardware. Estos sistemas paralelos se suelen utilizar cuando se tienen **volúmenes de datos grandes, algoritmos complejos y aplicaciones en tiempo real**.

1.3 ¿Como mejorar el rendimiento?

Existen tres formas principalmente:

- **Usar hardware mas rápido (trabajar más duro):** Si bien en la historia de la computación este método se pudo emplear, lo cual se refleja en la Ley de Moore, cada vez es mas difícil por límites físicos concretos. Estos límites respecto al crecimiento de la potencia informática efectiva se ven representados por los siguientes muros:
 - **Memory wall:** La velocidad de reloj del procesador aumentó más rápido que la velocidad de reloj de la memoria. Esto hace que la tecnología de la memoria este atrasada, lo cual se solventa con el uso de la memoria cache, pero no lo resuelve por completo.
 - **Power wall:** Los procesadores mas rápidos consumen mas energía. Los centros de computo están limitados por la potencia eléctrica total instalada y la potencia de refrigeración/extracción.
 - **ILP Wall:** Las dependencias de datos entre instrucciones y los branches en tiempo de ejecución limitan la cantidad de paralelismo que se puede aplicar a nivel de instrucción (ILP).

Otra opción es la computación cuántica, la cual hoy en día no esta lo suficientemente avanzada como para resolver los problemas que requerimos.

- **Usar algoritmos optimizados (trabajar más inteligentemente):** Técnicas que permiten optimizar el rendimiento:
 - Aprovechando el funcionamiento de la **memoria cache**.
 - **Minimizando el espacio de almacenamiento.**
 - **Minimizando las comunicaciones.**
 - **Minimizando la sincronización.**
 - **Optimizando el computo/computación.**

- **Usar mas hardware (pedir ayuda y sumar voluntades):** Sumar capacidad de computo, es decir, utilizar **arquitecturas paralelas**. Por ej.: Multiprocesadores, multicores, manycores (GPU, Xeon PHI), Clusters, Grid.

1.4 Tipos de paralelismo

1.4.1 Paralelismo implícito

El control y el procesamiento es **transparente al usuario**. Básicamente, **el programador no conoce los detalles de la funcionalidad**.

Las dos formas mas habituales son (pueden combinarse):

- **Pipelines:** Paralelismo a nivel de introducción (ILP). Dividir una función a realizar en etapas independientes. Se pueden utilizar arquitecturas escalares (ejecuta una instrucción por ciclo), superescalares (ejecuta más de una instrucción por ciclo), etc.
- **Division funcional:** utilizando varias unidades funcionales. Varias ALUs. Ejecutar simultáneamente varias instrucciones.

Generalmente, el mismo se da a **bajo nivel** (optimizaciones del compilador, entre otras técnicas), ya que el alto nivel introduce overhead por gestión.

1.4.2 Paralelismo explicito

El programador es el encargado de implementar la lógica del programa paralelo. Se parte de un programa secuencial el cual se quiere optimizar, en este sentido existen dos opciones:

- **Paralelizar el algoritmo secuencial.**
- **Diseñar un algoritmo paralelo distinto al secuencial.** La mejor solución puede no ser paralelizar el algoritmo secuencial, es por ello que se puede diseñar un algoritmo paralelo que resuelva el mismo problema con una estrategia distinta.

En cualquiera de los casos se debe analizar el comportamiento de las aplicaciones para saber como obtener la mejor solución.

Se debe considerar el comportamiento de las aplicaciones a paralelizar:

- **Intensivas en cómputo (CPU Bound):** llevan al límite a la CPU. Por ej.: La mayoría de las aplicaciones de cómputo científico.
- **Intensivas en memoria (Memory Bound):** llevan al límite el sistema de memoria. Por ej.: Ordenación y Búsquedas.
- **Intensivas en Entrada-Salida (E/S Bound):** llevan al límite el sistema de E/S. Por ej.: Simulación y Big data.
- **Híbridos:** La aplicación presenta distintas fases intensivas.

Áreas donde se requiere de mucho computo es útil usar sistemas paralelos. Se utilizan solo cuando son necesarios: cuando hay un problema de performance.

1.4.3 Comportamiento de las aplicaciones

Distintos eventos relacionados al cómputo requieren de un tiempo específico. Dado que las diferencias entre los eventos pueden ser despreciables, si las escalamos a todas de la misma forma obtendremos en cuenta que tipo de operaciones buscaremos evitar:

Evento	Tiempo Real	Tiempo Escalado
1 ciclo de CPU	0.3 ns	1 s
Acceso a cache de Nivel 1	0.9 ns	3 s
Acceso a cache de Nivel 2	2.8 ns	9 s
Acceso a cache de Nivel 3	12.9 ns	43 s
Acceso a RAM	120 ns	6 min
E/S a SSD	50 – 150 μ s	2-6 días
E/S a HDD	1 – 10 ms	1-12 meses
Internet: San Francisco a Nueva York	40 ms	4 años
Internet: San Francisco a UK	81 ms	8 años
Internet: San Francisco a Australia	183 ms	19 años
Reboot de SO de VM	4 s	423 años
Reboot de SO físico	5 m	32 milenios

1.4.4 Sistemas Operativos

Un sistema operativo que ejecute programas paralelos debería ser dedicado. Si bien no existen SO dedicados para cómputo paralelo.

Una característica deseable de un sistema operativo es que realice una adecuada asignación (proceso/unidad de procesamiento) para mantener la carga balanceada.

Alternativas:

- Distribuciones de SO conocidas y adaptarlas.
- Distribuciones de SO conocidas ya adaptadas con software paralelo.
- Distribuciones de SO más Robustas/Dedicadas que permiten desplegar un sistema paralelo que provee aplicaciones para el usuario final.

1.4.5 Lenguajes de Programación

Se suele programar en C. O wrappers que traducen a código C.

1.5 Modelos de programación paralela

El desarrollo de aplicaciones paralelas se basa en dos modelos de programación:

- **Memoria Compartida:** Procesos/Hilos se comunican mediante la memoria compartida. Herramientas de desarrollo: Pthreads, OpenMP, CUDA, etc.
- **Memoria distribuida:** Procesos/Hilos se comunican mediante mensajes. Herramientas de desarrollo: MPI, PVM, etc.

A partir de los mismos, se puede utilizar un **Modelo híbrido** combinando herramientas de ambos modelos. Por ej.: MPI + OpenMP + CUDA.

El modelo de programación paralela elegido **depende del modelo de arquitectura paralela disponible:**

PONER CUADRO

1.6 Características del desarrollo de aplicaciones paralelas

- **No determinismo:** Un programa paralelo es un programa concurrente y como tal, su ejecución puede arrojar más de un resultado.
- **Sincronización:** El buen uso de la sincronización (por exclusión mutua y/o por condición) ayuda a descartar los resultados no válidos y quedarnos con aquellos resultados que nos interesan.

El uso adecuado de la sincronización permite descartar deadlocks, livelocks, idling, race conditions y otros efectos no deseados.

- **Corrección y Depuración:** Programar un algoritmo paralelo requiere un desarrollo cuidadoso y de una depuración del programa paso a paso.
- **Fallas:** A mayor cantidad de hardware utilizado mayor posibilidad de falla. Se incrementa la complejidad de detectar el punto de falla.
- **Comunicaciones:** Al usar de redes de datos, los costos de comunicación son generalmente altos con respecto al tiempo de cómputo. Por esta razón, es necesario minimizar el costo de las comunicaciones.
- **Balance de carga:** Lograr distribuir equilibradamente el trabajo entre las unidades de procesamiento del sistema paralelo (mapeo), de modo de lograr que los tiempos de fin de trabajo sean similares.
- **Heterogeneidad:** Puede existir heterogeneidad en varios niveles: Hardware, Sistemas Operativos, Redes y Herramientas.
- **Rendimiento y Escalabilidad:** El objetivo del algoritmo paralelo es obtener el máximo rendimiento posible respecto al algoritmo secuencial. Se pueden emplear métricas para cuantificar ese rendimiento (Speedup, Eficiencia). Es deseable que la solución paralela sea escalable.
- **Portabilidad:** Es fácil lograr **portabilidad del código**, es decir establecer estándares que permitan ejecutar nuestro código en distintas arquitecturas. Sin embargo, es difícil lograr **portabilidad de rendimiento**, es decir lograr que el programa alcance en diferentes arquitecturas un rendimiento similar.
- **Seguridad:** Si el hardware paralelo tiene acceso desde y hacia el exterior es necesario incluir políticas de seguridad necesarias para evitar ataques.
- **Asincronismo:** El concepto de tiempo único-universal cambia. Aparecen distintos relojes en juego. La ocurrencia de un evento puede ser conocido por dos procesos en instantes distintos.

1.7 High Performance Computing (HPC)

Práctica de sumar potencia computacional con el fin de alcanzar mayor rendimiento en la ejecución de programas que resuelven problemas de gran tamaño de la ciencia, ingeniería o negocios.

HPC abarca el estudio de distintas áreas como: Simulación, Arquitecturas, Administración de recursos, Monitorización, Programación Paralela, I/O de alto rendimiento, Tolerancia a Fallas (Resiliencia), entre otras.

2 Memoria

2.1 Limitaciones del Sistema de memoria

En muchas aplicaciones la limitación no está en la potencia de cómputo del procesador sino en el sistema de memoria.

Dos parámetros esenciales del sistema de memoria son:

- **La latencia:** tiempo desde el “memory request” hasta que los datos están disponibles.
- **El ancho de banda:** velocidad con la cual el sistema de memoria puede depositar los datos en el procesador.

Dado que los procesadores evolucionaron más rápido que el sistema de memoria, el procesador realiza varias instrucciones por ciclo pero traerse operando puede tardar más de un ciclo, y en ese tiempo el mismo queda ocioso. De allí la importancia de la utilización de la memoria cache.

2.2 Memoria Cache

Memoria de rápido acceso (**mas rápida que la RAM**) y de poca capacidad (**menor que la RAM**). La misma permite mejorar la eficiencia puesto que tiene **menor latencia** que la memoria RAM, a costa de **ser más costosa**. Ante una *memory request*, primero se busca el dato en la memoria cache, luego pueden existir dos caminos:

- **Cache miss:** el dato no se encuentra en cache y debe traerselo de memoria RAM.
- **Cache hit:** el dato se encuentra en cache, se mejora el tiempo de respuesta (latencia).

El objetivo es establecer estrategias para minimizar la cantidad de accesos a memoria RAM maximizando la cantidad de accesos a memoria cache. De hecho, la memoria cache surge como consecuencia del principio de localidad.

2.2.1 Principio de localidad

Los accesos a memoria de un determinado programa no están distribuidos por todo el espacio de direcciones, sino que temporalmente se concentran en posiciones contiguas.

El mismo se manifiesta en dos aspectos:

- **Localidad temporal:** Un dato accedido recientemente es muy probable que sea accedido nuevamente en un futuro próximo.
- **Localidad espacial:** Un dato que se encuentra en una posición cercana a un dato recientemente usado es muy probable que se acceda en un futuro próximo.

Este principio se utiliza en la jerarquía de memoria para mejorar el rendimiento, principalmente al traer más de un dato cuando existe un miss en caché.

Evitar la siguiente confusión: El acceso a cualquier parte de la memoria RAM tiene el mismo costo (por ser de acceso aleatorio), por lo tanto, **el principio de localidad no se aplica al acceso a la memoria RAM**, se aplica cuando existe un miss en caché y en lugar de solo traer el dato que no se encuentra en caché se trae los datos cercanos que serán accedidos según el principio de localidad. Misma idea con el disco duro y la memoria RAM.

2.2.2 Estados y Niveles de la memoria Cache

La memoria cache puede estar en dos niveles:

- **Cold cache:** vacía o con datos irrelevantes.
- **Hot cache:** con datos relevantes, y todas las lecturas del programa son satisfechas por la cache.
- **Warm cache:** en una cache con 3 niveles: L1 hot, L3 cold y L2 en un estado intermedio.

2.2.3 Problemas de cache

- **Contención:** conflicto en el acceso a un recurso compartido en la jerarquía de memoria¹. En forma general, este problema surge debido a múltiples hilos intentan usar el mismo recurso compartido. Algunos problemas los puede solucionar el SO, aunque el programador puede ayudar.

Por ej.: Varios hilos comparten una misma memoria cache y la misma esta llena, uno de los hilos requiere traer un dato a la misma, y por ello, reemplaza un dato que otro hilo requerirá otro hilo.

Soluciones:

- Crear menos hilos.
 - Particionar la memoria cache.
 - Distribuir los hilos.
 - Rediseñar el algoritmo utilizado para obtener patrones de acceso libres de contención.
- **Coherencia:** Varios procesos/hilos ven la memoria compartida a través de diferentes cachés, uno de ellos ve un valor actualizado en su caché mientras que otros todavía ven el antiguo². Es un problema de hardware independiente de la sincronización del programa.

Por ej.: varios procesos actualizan un variable atómicamente.

Principalmente existen dos protocolos (protocolos hardware) de coherencia³:

- **Sistemas basados en bus (snoopy):** continuamente se observa si hay modificaciones en las líneas de cache.
- **Sistemas basados en diccionario:** un diccionario almacena la validez de las líneas de cache.

Existen dos formas principales de solucionar el problema (políticas):

- **Basadas en invalidación (MSI, MESI, MOSI, MOESI):** cuando cambia el valor de una variable almacenada en cache, se **invalidan** el resto de líneas de cache de toda unidad de procesamiento que tengan una copia de esa variable vieja. Generalmente, la mas utilizada.
- **Basadas en actualización (Dragon):** cuando cambia el valor de una variable almacenada en cache, se **actualizan** las líneas de cache de toda unidad de procesamiento que tengan una copia de esa variable vieja. Menos utilizada debido a que si el resto no vuelve a necesitar el dato, el mismo se escribe sin sentido.

2.2.4 Optimización utilizando principio de localidad

Uno de los problemas más típicos a solucionar con el principio de localidad es la multiplicación de matrices. Dado que la misma realiza multiplicaciones de filas de una matriz, con columnas de la otra matriz, es conveniente almacenar los datos de la segunda matriz operando en forma de columnas, y recorrer los datos en columnas ayudara a tener menos cache miss, puesto que al traerse un dato de una columna se traerá mas datos de la misma columna, las cuales se utilizaran en un instante próximo.

2.2.5 Minimizando el espacio de memoria

Cuando utilizamos una matriz cuadrada triangular, los elementos por encima o por debajo de la diagonal son cero, por lo tanto, no imponen una información importante. Cuando los requerimientos de memoria son críticos y estas matrices son muy grandes, no es necesario almacenar una gran cantidad de ceros. La idea es minimizar el espacio de almacenamiento evitando guardar los elementos de la parte nula de la matriz. Esto modifica la forma en la cuál accedemos a la misma.

¹RAM, unidad de disco, memoria caché, buses internos o dispositivos de red

²En un sistema con un procesador y una unidad de procesamiento, no hay interferencias debido a que múltiples procesos/hilos se ejecutan de a uno a la vez en el único procesador y ven la memoria a través de la misma jerarquía.

³Formas de detectar la falta de coherencia.

Respecto a la multiplicación de este tipo de matrices, podemos realizar una nueva optimización: Sólo recorreremos las posiciones que no son cero.

Si bien minimizamos el espacio de memoria, tenemos un trade-off:

- Ganamos espacio de almacenamiento
- La cantidad de cálculos es mayor por lo tanto podría perderse en rendimiento

Para saber si el trade-off es positivo, se debe analizar empíricamente.

3 Clasificación de arquitecturas paralelas

3.1 Por su organización lógica

Organización desde el punto de vista del programador.

3.1.1 Mecanismos de control (Taxonomía de Flynn)

Los mecanismos de control son formas de expresar las tareas paralelas. La **taxonomía de Flynn** es el mecanismo de control más conocido. El mismo se basa en dos aspectos:

- **El flujo de instrucciones concurrentes:** secuencia de instrucciones que la unidad de control despacha durante un ciclo de instrucciones (puede ser simple o múltiple).
- **El flujo de datos:** flujo de datos sobre el que operan las instrucciones (puede ser simples o múltiples).

En este sentido podemos clasificar una arquitectura cómo:

- **Single Instruction Single Data (SISD):** Un único flujo de instrucciones y un único flujo de datos.

Una única unidad de procesamiento ejecuta un único flujo de instrucciones. Instrucciones ejecutadas en secuencia, una por ciclo.

Por ej.: Monoprocesadores.

- **Single Instruction Multiple Data (SIMD):** Un único flujo de instrucciones y múltiples flujos de datos.

Conjunto de unidades de procesamiento idénticas que ejecutan la misma instrucción, sincrónicamente, sobre distintos datos. La unidad de control hace broadcast de la instrucción a todas las unidades de procesamiento.

Por ej.: GPU, SSE, AVX.

- **Multiple Instruction Single Data (MISD):** Múltiples flujos de instrucciones y un único flujo de datos.

Conjunto de unidades de procesamiento que ejecutan distintas instrucciones (cada instrucción despachada por una unidad de control diferente), sincrónicamente, sobre el mismo flujo de datos.

Por ej.: Hacking de un mensaje codificado.

- **Multiple Instruction Multiple Data (MIMD):** Múltiples flujos de instrucciones y múltiples flujos de datos.

Conjunto de unidades de procesamiento que ejecutan, asincrónicamente, diferentes instrucciones sobre diferentes flujos de datos. Pueden ser de memoria compartida o distribuida.

Por ej.: Multicores, Clusters.

Existen extensiones de la taxonomía:

- **SPMD (Single Program Multiple Data - Un programa, múltiples datos):** Conjunto de unidades de procesamiento que trabajan simultáneamente sobre el mismo conjunto de instrucciones del mismo programa (aunque en puntos independientes) y operan sobre datos diferentes.
- **MPMD (Multiple Program Multiple Data - Múltiples programas, múltiples datos):** Conjunto de unidades de procesamiento que trabajan simultáneamente sobre al menos dos programas independientes.

3.1.2 Modelos de comunicación

Los modelos de comunicación son mecanismos para especificar la interacción entre tareas. Se pueden clasificar en dos grandes grupos:

- **Memoria compartida:** todos los procesadores comparten la memoria.
 - **UMA (Uniform Memory Access):** La memoria física se comparte de manera uniforme por todos los procesadores. El costo de acceso a memoria es el mismo para todos los procesadores. La interconexión es a través de un bus.
 - **NUMA (Non-Uniform Memory Access):** La memoria física se distribuye entre todos los procesadores, donde cada uno tiene su propia memoria local. Todas estas memorias locales forman un gran espacio de direcciones, donde el costo de acceso a una memoria remota es mayor que el acceso a memoria local.

En este tipo de arquitectura:

- Expansión limitada.
 - Un fallo de un procesador afecta a todo el sistema.
 - Aumentar la cantidad de procesador tiende a aumentar la contención de memoria.
- **Memoria distribuida:** Procesadores conectados por una red de interconexión. Cada procesador tiene su propia memoria local (solo pueden acceder a ella). La interacción entre procesadores es sólo por pasaje de mensajes.
 - Expansible a bajo costo.
 - Fallos en una CPU pueden no afectar a todo el sistema.
 - La comunicación es más lenta.

Es posible tener ambos modelos (modelo híbrido).

Grados de acoplamiento de los procesadores:

- **Fuertemente acoplados:** Procesadores y red físicamente cerca. Mas costosos. Baja latencia. Tasas de transferencia alta. Mas eficientes energéticamente.
Por ej.: Multicores, Manycores.
- **Débilmente acoplados:** Procesadores y red físicamente lejos. Menos costosos. Alta latencia. (Variables). Tasas de transferencia baja. (Variables). Menos eficiente energéticamente.
Por ej.: Cluster, Grids.

3.2 Por su organización física

Organización desde el punto de vista del hardware.

3.2.1 Espacio de direcciones

Relacionado con el modelo de comunicación.

- Espacio de direcciones compartido por todos los procesadores. Memoria compartida.
- Espacio de direcciones local a cada procesador. Memoria distribuida.

3.2.2 Red de interconexión

Cómo se interconectan los nodos: dispositivo que quiera conectarse a la red de interconexión.

Las redes de interconexión se pueden clasificar en:

- **Redes estáticas:** su topología queda establecida en forma definitiva y estable cuando se instala el sistema.
Por ej.: Lineal, Anillo, Conexión total, Malla, Toro, Estrella, Hipercubos, Jerárquicas, Jerárquicas Fat-Tree, etc.
- **Redes dinámicas:** su topología puede variar durante la ejecución de los procesos.
 - **Buses:** Todos los procesadores acceden al bus para intercambiar datos. Simple. Una transferencia a la vez. Debe existir un nodo que arbitre la comunicación.
Distancia entre nodos $O(1)$, broadcasting fácil, el costo escala linealmente con la cantidad de nodos.
Ancho de banda limitado, alta latencia, bloqueante.
 - **Crossbar:** Conecta P procesadores con M bancos de memoria a través de PxM switches. La posición del switch cambia dinámicamente.
No bloqueante, acceso rápido y constante, fácilmente escalable.
El costo crece en orden PxM, escalabilidad costosa.
 - **Multistage:** trata de encontrar el mejor compromiso entre los buses y los crossbars.

3.2.3 Granularidad

Relacionado al número y la potencia de los procesadores.

- **Grano fino:** Muchas unidades de procesamiento poco potentes.
- **Grano grueso:** Pocas unidades de procesamiento muy potentes.
- **Grano medio:** Equilibrio entre los anteriores. Es la arquitectura que tenemos en las computadoras de nuestras casas.

Nota: no confundir con la granularidad en software (volumen de carga de trabajo por tarea). Aunque la granularidad de hardware (la arquitectura utilizada) se decide dependiendo de la de software.

4 Evolución de las arquitecturas paralelas

4.1 Paralelismo en procesadores

- **Monoprocesador:** procesador o CPU que posee una única unidad de procesamiento.
Paralelismo a nivel de instrucción (ILP).
- **Multiprocesadores:** más de una unidad de procesamiento. Dos hilos en una UP NO es paralelismo.
- **Cluster/Multiclusters:** Un cluster son varias máquinas interconectadas mediante una red de comunicación que se comportan como si fuesen una única máquina permitiendo mayor potencia de cómputo.

Es posible conectar varios clusters formando una arquitectura multicluster. Pueden ser Homogéneos o Heterogéneos.

- Mayor posibilidad de fallas.
- Mayor latencia.
- Gestión y administración compleja.

- **Multicores:** un procesador o CPU que en un mismo circuito integrado incorpora dos o más unidades de procesamiento/ejecución independientes conocidos como cores o núcleos.

Pueden clasificarse como:

- **Homogéneos/Simétricos:** Cores idénticos.
- **Heterogéneos:** Cores diferentes.
 - * **Heterogéneos pp:** Compilar código para cada tipo de core.
 - * **Asimétricos:** Misma Instruction Set Architecture. Una única compilación de código. Por ej.: CPU + GPU + NPU.
- **Grid:** sistema que permite compartir recursos distribuidos geográficamente. Sumar equipos no utilizados al 100% para procesar aplicaciones con alta demanda computacional y gran carga de datos.
- **Manycore:** arquitectura que posee un gran número de cores simples. GPUs.
- **Cloud:** paradigma, basado en el concepto de virtualización, que ofrece servicios (orientado a servicio) a través de Internet con un alto grado de transparencia. Un Cloud provee aplicaciones comunes de negocio on-line accesibles desde un navegador web, mientras el software y los datos se almacenan en los servidores.
No confundir con Grid asociado a la computación distribuida y paralela, donde varios nodos débilmente acoplados actúan en conjunto para resolver una tarea muy grande.
 - **Cloud Público:** Empresas proveedoras de servicios Cloud. Amazon, Google, IBM, Azure.
 - **Cloud Privado:** Infraestructura de cloud propia. Software para montar un Cloud: Open-Stack, Eucalyptus, Proxmox.
 - **Cloud Híbrido:** Cloud Privado que utiliza servicios de Cloud Público ante picos de demanda.
- **Computación Cuántica:** computadora que aproveche algunos principios de la mecánica cuántica.

5 Diseño de algoritmos paralelos (estrategias de Ian Foster)

En el área de la Ingeniería de Software se proponen varios modelos para desarrollo de software. Cualquiera sea el modelo elegido, la ingeniería de software nos destaca la importancia de realizar un buen diseño antes de llevar a cabo cualquier implementación.

En particular, **no nos podemos abstraer demasiado del hardware a utilizar**, ya que es importante obtener el mayor beneficio de este para mejorar el rendimiento.

5.1 Descomposición o Particionado

Identificar el trabajo que puede hacerse en paralelo dividiéndolo en tareas.

- Se trata de definir un gran número de tareas para tener mayor flexibilidad frente a potenciales algoritmos paralelos.
- No se tienen en cuenta en esta etapa la cantidad de UP y aspectos específicos de la máquina destino.
- Las tareas no son procesos o hilos, son cosas a hacer.

Puede existir dependencia entre tareas o no⁴

Existen los siguientes enfoques:

- **Descomposición funcional:** Se descompone el problema por funcionalidad. Reduce la complejidad del diseño general: puede plantearse la solución como bloques de código simples que interconectados funcionen como un único bloque complejo. Generalmente las tareas a realizar son distintas por cada funcionalidad.
- **Descomposición de datos o de dominio:** Se enfoca principalmente en los datos a tratar. Generalmente las tareas son iguales o similares, y actúan sobre distintos datos.
- **Descomposición mixta:** Se aplica una descomposición funcional macro con una descomposición de datos por cada función.

5.1.1 Descomposición recursiva

La descomposición recursiva se utiliza en problemas que siguen una estrategia “divide y vencerás”. El problema se descompone en un par de subproblemas idénticos independientes.

Por ej.: Ordenación (Quicksort, Mergesort), Reducción (dado N valores aplicar una función para obtener un único valor), etc.

5.1.2 Descomposición basada en los datos

La descomposición basada en datos identifica el conjunto de datos sobre los cuales se realizarán los cálculos. Luego divide los datos entre tareas.

Existen distintas alternativas basadas en:

- **Datos de salida:** Si los cálculos de los resultados de un programa paralelo son independientes, la descomposición natural es una tarea por resultado.

Por ej.: multiplicación de matrices, mapeo (aplicar una misma función a distintos tipos de datos).

- **Datos de entrada:** Cada tarea se asocia con una partición de los datos de entrada. Generalmente, se requiere un paso posterior para recuperar los resultados.

Por ej.: Búsqueda del valor máximo o mínimo en una lista, Ordenación, etc.

Problema de la frontera: Las tareas comparten áreas de datos.

⁴Cuando las tareas tratan con conjuntos de datos independientes, hay poca o ninguna dependencia y no hay necesidad de comunicación de datos o resultados, el problema se conoce como **Embarrassingly parallel**.

- **Datos de entrada y salida:** Se combinan las anteriores estrategias.
Por ej.: buscar la frecuencia con que se repiten ciertas sílabas en un listado de palabras.
- **Datos intermedios:** El problema se descompone en etapas sucesivas que a su vez generan dependencias entre tareas.
Por ej.: multiplicación de matrices pensada en dos etapas: multiplicar celdas y sumar tales multiplicaciones para obtener los valores finales.

5.1.3 Descomposición exploratoria

Se aplica a problemas que evolucionan durante la ejecución explorando un espacio de soluciones posibles. La descomposición progresa con la ejecución y genera **tareas de forma dinámica**.

Por ej.: N-reinas, Puzzle-15, etc.

A los algoritmos que resuelven este tipo de problemas se les conoce como algoritmos **Branch and Bound**: El algoritmo explora las ramas (branch) del árbol. Una rama se compara con los límites (bound) estimados superior e inferior de la solución óptima, y se descarta si no puede producir una solución mejor que la mejor encontrada hasta ahora por el algoritmo.

Se pueden elegir varios criterios para finalizar con la exploración:

- Cuando se encuentra la primer solución.
- Cuando se encuentra una solución óptima.
- Cuando se encuentra una solución sub-óptima.
- Cuando se encuentran todas las soluciones posibles.

Suelen ser superlineales pues: Tareas no finalizadas pueden terminar de inmediato cuando se alcance una solución deseada. La carga de trabajo puede ser muy distinta entre la solución secuencial y la solución paralela.

5.1.4 Descomposición especulativa

Se utiliza en problemas donde el próximo paso es una acción entre varias posibles, que sólo puede determinarse cuando las tareas precedentes concluyan.

Por ej.: Simulación discreta (incendios, inundaciones etc), movimiento de robots en un espacio limitado, etc.

5.1.5 Descomposición híbrida

Las técnicas de descomposición pueden combinarse. La descomposición puede estructurarse en múltiples etapas y en cada una aplicar una estrategia de descomposición diferente.

5.2 Comunicación

Determinar los patrones de comunicación (y/o sincronización) entre las tareas identificadas en el paso previo.

La comunicación es fuente de overhead y **debe ser minimizada**.

- **Global/Local:** Las tareas se comunican sólo con tareas vecinas (local) o con varias tareas (global).
- **Estructurado/No estructurado:** Patrones regulares (lista, árbol, grilla) o irregulares (grafos).
- **Estático/Dinámico:** El patrón no cambia en ejecución ó no.
- **Síncrono/Asíncrono:** Comunicación entre emisor y receptor coordinada (PMS), ó Comunicación entre emisor y receptor sin coordinación (PMA).

- **Solo Lectura/Lectura-Escritura:** Una tarea sólo lee datos asociados a otra tarea ó también los escribe.
- **Unidireccional/Bidireccional:** en un único sentido o ambos.

5.3 Aglomeración

Las tareas y las comunicaciones identificadas en pasos anteriores se combinan en tareas de mayor tamaño para adaptarlas al hardware paralelo disponible.

- El algoritmo que surge de las etapas previas es aún abstracto, es decir NO está especializado para la ejecución eficiente en una máquina paralela particular.
- Puede ser ineficiente si existen más tareas que unidades de procesamiento disponibles.
- El objetivo de esta etapa es obtener un algoritmo que se ejecute en forma eficiente sobre una máquina paralela real. Por lo tanto, pueden descartarse posibilidades detectadas anteriormente.

Dado que se busca eliminar el overhead introducido por la comunicación, una forma de reducirlo es:

- **Incrementar la granularidad** asociando más cómputo por tarea sin atentar contra la localidad de los datos.
- **Replicar el cómputo** cuando la comunicación es más costosa que el propio cálculo.

5.4 Mapeo

Asignar cada tarea a una unidad de procesamiento para maximizar la utilización del mismo y reducir la comunicación.

Un mapeo adecuado debe asegurar:

- Un correcto balance de carga entre unidades de procesamiento (volumen de trabajo uniforme entre tareas).
- Evitar esperas ociosas (Idling).

5.4.1 Mapeo estático

Las tareas son distribuidas entre las unidades de procesamiento antes de la ejecución. Suele ser útil cuando se conoce a priori la cantidad de tareas a ejecutar y su carga de trabajo.

La elección de una buena estrategia de mapeo estático depende de varios factores:

- Conocimiento de las tareas.
- Tamaño de los datos asociados a cada tarea.
- Las características de interacción entre tareas.
- El paradigma de programación paralela.

Existen tres estrategias de mapeo estático:

- **Basada en partición de datos:** Se deriva de una descomposición basada en datos. Existen dos formas de representar datos en un algoritmo:
 - **Arreglos:**
 - * **Bloques:** se utiliza una distribución por bloques distribuyendo porciones del arreglo de forma de aprovechar la localidad de datos (arreglos por bloques).
 - * **Cíclicos:** se utiliza cuando el volumen de trabajo se modifica en tiempo de ejecución.
 - **Grafos:** algoritmos que operan sobre estructuras de datos dispersas (grafos) que no son regulares como los arreglos. La idea es particionar el grafo de manera de distribuir los vértices a procesar de manera “balanceada” entre las unidades de procesamiento.

- **Basada en partición de tareas:** Se puede utilizar esta técnica cuando el cálculo puede expresarse naturalmente en forma de un grafo con dependencias estáticas de tareas de tamaño conocido. Se tiene un grafo de tareas, NO de datos.
- **Jerárquico:** grafo con dependencias de tareas en forma de árbol binario.

5.4.2 Mapeo dinámico

Se utiliza mapeo dinámico en situaciones donde:

- El mapeo estático lleva a una distribución de trabajo desbalanceada entre unidades de procesamiento (Por ejemplo: al utilizar arquitecturas heterogéneas).
- El grafo de dependencias de tareas es por naturaleza dinámico.

Un mapeo dinámico puede generar overhead pero se compensa el balance de carga.

Las estrategias clasifican en:

- **Centralizadas:** Todas las tareas a realizar se mantienen en una de dos posibles formas: Un proceso ocioso obtiene tareas de la estructura de datos centralizada **ó** solicitándolas al Master. Esta estrategia puede tener problemas de escalabilidad a medida que se incrementa el número de procesos a utilizar: un gran número de accesos a la estructura centralizada o al proceso Master genera cuellos de botella.
- **Distribuidas:** todas las unidades de procesamiento tienen tareas asignadas, y se las van transfiriendo entre sí.

Se debe tener en cuenta:

- **¿Cuánto trabajo se transfiere?** puede implicar mucha comunicación o procesos sin trabajar.
- **¿Cómo se transfiere el trabajo?** directo (un proceso decide a quien enviar trabajo), por demanda (un proceso solicita a otro) y work stealing (un proceso le roba a otro una tarea).
- **¿Cuándo se transfiere el trabajo?** La solicitud de trabajo podría llegar cuando el proceso está trabajando, esto implica una demora en el proceso que requiere ese trabajo.

6 Modelo de programación sobre memoria compartida

En un modelo de programación sobre memoria compartida, los procesos o hilos comparten la memoria y se comunican mediante variables.

En este tipo de modelos pueden existir problemas de **condición de carrera**: . Para solucionarlos:

- **Exclusión mutua.**
- **Sincronización por condición.**

El modelo de programación sobre memoria compartida puede utilizarse en un modelo de arquitectura de memoria compartida con un único espacio de memoria direccionable. Sobre otros modelos de arquitectura, se requiere una capa de abstracción (SSI) que genera overhead impactando en el rendimiento.

6.1 No confundir

- Una variable compartida **NO** es una variable global.
- Un proceso o hilo **NO** es una función o un procedure.

6.2 Pthreads

Posee un conjunto de funciones que permiten:

- Gestionar hilos (threads).
- Sincronizar hilos: Mutexes, Variables condición, Barreras.
- Interactuar con la API Semaphore (no es parte del estándar).

6.2.1 Programa básico

Un programa Pthreads básico requiere de al menos 4 pasos:

1. **Definir los Hilos.**

```
pthread_t miHilo;
```

2. **Crearlos.**

```
int pthread_create(pthread_t *thread, const pthread_attr_t *attr, void *(*start_routine)(void*), void *arg);
```

3. **Implementar la función de comportamiento del Hilo.**

```
void* funcion(void *arg);
```

Generalmente el argumento apunta a una variable de un tipo simple. En el modelo de memoria compartida con Pthreads, asumimos que la función del comportamiento de un hilo recibe un id o una estructura para ids compuestos. NO recibe otros parámetros ni tampoco referencias a variables compartidas (estas ya se encuentran compartidas).

Los resultados de estas funciones se devuelven mediante variables compartidas, no en el `pthread_exit`, solo se retorna solo para control.

4. **El programa principal debe esperar que los hilos terminen su ejecución.** El programa C que se ejecuta y crea los hilos, actúa como un hilo más. El main, luego de crear los hilos, debe esperar a que estos finalicen. Para esto utiliza la función join cuyo prototipo es:

```
int pthread_join(pthread_t thread, void **value_ptr);
```

6.2.2 Sincronización

Pthreads permite la sincronización mediante mecanismos como:

- **Mutexes:** se utiliza para sincronización por **exclusión mutua**. Tiene dos posibles estados: bloqueado o desbloqueado. En caso de estar desbloqueado solo uno puede apoderarse del mutex. Se debe definirlo (compartido), inicializarlo, utilizarlo y destruirlo.
- **Variables condición:** se utilizan para **sincronización por condición**. Permiten detener la ejecución de un hilo a la espera de la ocurrencia de alguna condición. Cuando esa condición se cumple, algún otro hilo enviará una señal al hilo dormido para que continúe su ejecución.
Cada variable condición debe utilizarse **en conjunto con un mutex**.
Se debe definir las (compartidas), inicializarlas, utilizarlas y destruirlas.
- **Barreras:** hace que un conjunto de hilos/procesos se esperen (duerman) para poder continuar su ejecución.
Se debe definir las (compartidas), inicializarlas, utilizarlas y destruirlas.
Para el sistema operativo el costo de crear hilos una y otra vez es mayor que dormirlos y desperdiciarlos. Por lo tanto, si se quiere realizar algo en etapas, se deben utilizar barreras y no destruir los hilos y volver a crearlos.

Es posible implementar semáforos mediante la biblioteca semaphore pero no es parte del estándar.

6.2.3 Afinidad

La afinidad es elegir la unidad de procesamiento donde debe ejecutar un hilo.

Debe asegurarse que dos o más hilos no ejecuten sobre la misma unidad de procesamiento. De esto se encarga el Sistema Operativo, pero el programador podría querer una asignación diferente.

Esto puede realizarse con Pthreads.

6.3 OpenMP

OpenMP es una API para programación paralela sobre memoria compartida multiplataforma.

OpenMP sigue el modelo **Fork-Join**: un hilo (Master thread) viene ejecutando el código secuencialmente. En algún momento, se divide en T hilos (Fork), cada uno ejecuta su parte en forma paralela y luego se esperan (Join) en un punto a partir del cual se continua con la ejecución secuencial.

6.3.1 Estructura de control paralela

OpenMP utiliza la directiva `#pragma instruccion`, directiva del compilador de C.

`#pragma omp parallel` se utiliza para crear hilos (por defecto crea los máximos que puede, sino `omp_set_num_threads(n)` crea n).

Se puede activar o no el anidamiento de instrucciones de OpenMP, por defecto desactivado.

6.3.2 Trabajo compartido

OpenMP posee cuatro directivas para la distribución de trabajo entre hilos:

- **do/parallel do, omp for:** distribuye entre los hilos las iteraciones de una sentencia for.
Las políticas de distribución de iteraciones pueden ser:
 - **Estática (por defecto):** se distribuyen proporcionalmente entre los hilos.
 - **Dinámica:** se distribuyen por demanda y de a cierta cantidad (chunk).
 - **Guiada:** distribución de iteraciones variable.
 - **Runtime:** distribución que se indica desde el Sistema Operativo por medio de la variable de entorno `OMP_SCHEDULE`.

- **sections:** asigna a los hilos bloques de código consecutivo e independiente.
- **single:** especifica un bloque de código que será ejecutado por un único hilo. Existe una barrera implícita al final del bloque.
- **master:** similar a single. El bloque de código lo ejecuta sólo el Master thread. NO existe barrera al final del bloque.

6.3.3 Entorno de datos

En OpenMP, por defecto las variables son visibles por todos los hilos. Para gestionar el entorno de los datos, se agregan cláusulas a las directivas:

- **shared:** se comparten datos entre la región secuencial y la región paralela.
- **private:** la variable afectada por esta cláusula es privada a cada hilo. Cada hilo tiene una copia local privada, la cual usa como variable temporal, no se inicializa y su valor no se mantiene fuera de la región paralela.
- **firstprivate:** idem private pero el valor se inicializa con el valor original de la variable.
- **lastprivate:** dem firstprivate pero el valor finaliza con el último valor calculado.
- **reduction:** reduce el resultado de todos los hilos mediante alguna operación (-, +, *, min, max, etc.).

Se puede poner una cláusula `default(type)` donde se define el tipo por defecto.

6.3.4 Sincronización

OpenMP provee las siguiente cláusulas para implementar algunos mecanismos de sincronización:

- **critical:** encierra un bloque de código (región crítica) que debe ser ejecutado por un hilo a la vez.
- **atomic:** similar a critical pero encierra una sola instrucción.
- **ordered:** se utiliza cuando es necesario que las instrucciones se ejecuten según el orden de las iteraciones.
- **barrier:** para realizar una barrera explícita.
- **nowait:** para evitar la barrera implícita, especifica que los hilos que terminaron su trabajo puedan continuar sin esperar al resto.

6.3.5 Afinidad

Al igual que en Pthreads se puede manejar la afinidad.

7 Modelo de programación sobre memoria distribuida

En el modelo de programación sobre memoria distribuida cada proceso tiene su propia memoria local/privada. Un proceso no puede acceder al espacio de memoria de otro proceso. Los procesos se comunican y sincronizan mediante el envío y recepción de mensajes a través de una red de comunicación.

7.1 Message Passing Interface (MPI)

Message Passing Interface (MPI) es un estándar¹ que define la sintaxis y la semántica de funciones para pasaje de mensajes. No define los comandos, OpenMPI se encarga de implementar el estándar.

7.1.1 Funcionamiento general

Compilación:

- Para ejemplificar el funcionamiento de MPI, suponemos un cluster con varias máquinas conectas a una red de comunicación física y que usamos OpenMPI.
- Cada máquina debe conocer el archivo ejecutable (binario).
- MPI no compila automáticamente el archivo fuente ni distribuye el binario.
- El programador debe compilar el código fuente y distribuir el binario. La compilación depende de las características del cluster:
 - **Cluster homogéneo:** se compila una única vez y se distribuye el binario.
 - **Cluster heterogéneo:** se compila una vez por máquina.

Ejecución:

- Todas las unidades de procesamiento ejecutan una copia del mismo programa.
- El Runtime System de MPI automáticamente asigna un identificador único (rank) a cada proceso creado. Generalmente, lo asigna en el orden del archivo de máquinas.

7.1.2 Estructura de programa

Las cuatro funciones principales de control son:

- **MPI_init:** inicializa el ambiente de ejecución MPI. Recibe como parámetros los argumentos que recibió el programa C. Es la primera sentencia que debe ejecutarse después de la definición de variables.
- **MPI_Comm_rank:** permite obtener el identificador de proceso (0..P) asignado por el Runtime System de MPI.
- **MPI_Comm_size:** permite obtener el número de procesos creados.
- **MPI_Finalize:** termina la ejecución del ambiente MPI. Es la última función que debe ejecutarse antes de la sentencia return.

Varias funciones MPI reciben como parámetro el nombre de un comunicador: utilizado para agrupar procesos. Esto permite organizar mejor los procesos. **MPI_COMM_WORLD** es el comunicador por defecto que incluye a todos los procesos en la ejecución.

Todas las unidades de procesamiento ejecutan una copia del mismo programa. Existirá una copia del programa para cada proceso. Por lo tanto, existirá una copia de la variable para cada proceso: **VARIABLE LOCAL Y PRIVADA**.

7.1.3 Comunicación

- **Punto a punto:** Comunican sólo dos procesos. Un proceso hace un “tipo” de send y otro proceso hace un “tipo” de receive.

Una operación puede ser:

- **Bloqueante:** si por alguna razón la sentencia (send o receive) no retorna el control al programa.
 - **No Bloqueante:** si la sentencia retorna el control de inmediato independientemente de lo que ocurra con el mensaje.
 - **Sincrónica:** si el programa que envía no puede continuar hasta que no haya una recepción.
- **Colectivas:** Comunican varios procesos. Todos los procesos involucrados ejecutan la misma función de comunicación. Son **bloqueantes** pero **no** necesariamente **sincrónicas**. Se pueden utilizar barreras, broadcast, scatter, gather, reduce, allreduce, etc.

8 Modelo de programación híbrido

Las arquitecturas de memoria compartida y memoria distribuida pueden combinarse formando una arquitectura híbrida con mayor potencia de cómputo.

El modelo de programación híbrido sólo tiene sentido cuando contamos con una arquitectura híbrida compuesta por un cluster de máquinas multicore.

`MPI_Init_thread()` es una alternativa de MPI para “avisarle” al entorno que vamos a crear hilos y de esta forma está preparado para una ejecución “thread safe”.

8.1 Modularidad

Una forma de organizar el código híbrido es tener el código de cada herramienta por separado.

Razones para modularizar:

- Facilidad para depurar, mayor modularidad y portabilidad
- En ocasiones no contamos con el código fuente.

Restricciones:

- Algoritmos que tienen etapas donde los procesos se comunican en cada una de ellas no permiten modularidad.

9 Métricas

Utilizamos métricas para evaluar el comportamiento de un sistema. En un sistema paralelo, importa cual es la mejora respecto a la solución secuencial. Estas métricas están asociadas al **rendimiento global (throughput)**: cantidad de trabajo u operaciones que se puede hacer en un determinado lapso de tiempo (Flops, Mips, Mbps, etc.).

Para algoritmos secuenciales que resuelven el mismo problema, solemos calcular el comportamiento asintótico con Big O. Dado que en un sistema paralelo, a pesar de resolver el mismo problema y con el mismo comportamiento asintótico, los algoritmos pueden diferir en los tiempos de ejecución debido a otros factores como lo son la localidad, debemos utilizar otra forma de medir la mejora.

Recolección de muestras:

- Para obtener las métricas debemos ejecutar los algoritmos y obtener el tiempo de ejecución.
- Se debe tener en cuenta que dos o más ejecuciones del mismo algoritmo pueden arrojar tiempos de ejecución ligeramente diferentes. Para mitigar esta variabilidad, debemos correr el algoritmo varias veces y obtener el tiempo de ejecución promedio.
- existen algoritmos donde una sola ejecución puede llegar a demorar horas. Alternativas:
 - Se acepta ejecutarlo una única vez.
 - Estimar el tiempo de ejecución sin ejecutarlo (firma del código).

9.1 Speedup

El **Speedup** es una **métrica de rendimiento que representa el beneficio relativo alcanzado** al ejecutar un programa. Suponiendo un programa al cual se le hizo alguna mejora:

$$S = \frac{T_a}{T_b} \quad \begin{cases} T_a : & \text{tiempo de ejecución antes de la mejora,} \\ T_b : & \text{tiempo de ejecución después de la mejora.} \end{cases}$$

Si bien la anterior es la definición nos interesará cuando la mejora es la paralelización:

$$S = \frac{T_s}{T_p} \quad \begin{cases} T_s : & \text{tiempo de ejecución secuencial,} \\ T_p : & \text{tiempo de ejecución paralelo con } \mathbf{P} \text{ unidades de procesamiento.} \end{cases}$$

Si T_s es el mejor tiempo secuencial se trata del speedup **absoluto**, sino es el **relativo**.

Podemos analizar el valor del *speedup* en distintos escenarios:

- $S \leq 1$: No hay mejora al paralelizar. De hecho, puede ser peor que la versión secuencial debido al overhead introducido por el paralelismo. Otra posible causa es que el tamaño del problema sea chico.
- $S < P$: Hay una mejora respecto a la versión secuencial, aunque no se aprovechan completamente los P procesadores. El overhead no es tan alto, pero sigue presente.
- $S = P$: Se logra un Speedup lineal o perfecto. Es el caso ideal, aunque poco común en la práctica, ya que casi siempre hay algún tipo de overhead que impide alcanzar este valor.
- $S > P$: Se obtiene un speedup superlineal. Aunque suene contradictorio, esto puede ocurrir por varias razones:
 - **El algoritmo secuencial no es el más eficiente.** A veces, la versión paralela incluye optimizaciones que también podrían haberse aplicado al algoritmo secuencial, pero no se hizo. En ese caso, el speedup es relativo.
 - **La versión paralela hace menos trabajo que la secuencial.** Por ejemplo, en una búsqueda sobre un grafo, la versión secuencial explora todas las ramas, mientras que en la paralela, un proceso puede encontrar la solución y avisar al resto para que detengan su búsqueda.

- **Beneficios por la arquitectura.** Al usar varios procesadores, se dispone de mayor capacidad de caché. Esto permite que los datos que no entraban en la caché de una sola CPU sí lo hagan al repartirse entre varias, mejorando el rendimiento.

9.2 Eficiencia

La **eficiencia** es una **métrica de rendimiento** que indica el **porcentaje de tiempo en el que las unidades de procesamiento están realizando trabajo útil**. Básicamente, indica cuanto se está usando cada UP.

$$E = \frac{S}{P}$$

Generalmente, $E \in (0, 1)$.

9.3 Overhead

El overhead es:

$$T_O = P \cdot T_p - T_s$$

Generalmente, el overhead crece de forma:

- **Linealmente con las UPs:** Cada proceso/hilo introduce una parte intrínsecamente lineal no paralelizable. Además, agregar más UP implica más aumento de la comunicación y desbalance de carga.
- **Sublinealmente en función del tamaño del problema/carga de trabajo (W):** Esto implica que T_O será menor cuanto mayor sea W . Este crecimiento sublineal se tiene en cuenta en la ley de Gustafson.

9.4 Balance de carga

Existen situaciones en que se producen desbalances de carga: procesos o hilos que realizan mayor (o menor) cantidad de trabajo que otros.

El balance de carga es un porcentaje que se mide como:

$$B = \frac{\text{Promedio}(T)}{\text{MAX}(T)}$$

9.5 Escalabilidad en los sistemas paralelos

9.5.1 Ley de Amdahl

“A menos que prácticamente todo un programa secuencial esté paralelizado, el speedup posible va a estar muy limitado independientemente de las unidades de procesamiento disponibles”

En palabras sencillas, la ley de Amdahl indica que existe una parte del programa no paralelizable que limita el speedup máximo que se puede conseguir para un tamaño de problema fijo. Esto implica que a pesar que aumentemos de forma infinita las unidades de procesamiento, obtendremos un speedup máximo. Si f es la fracción no paralelizable, no podemos obtener una aceleración mejor que $\frac{1}{f}$.

- Supone que los requerimientos se mantendrán invariantes en el tiempo.
- Supone un tamaño de problema fijo.
- Útil para ciertas aplicaciones con volumen de datos bajo no variables en el tiempo. Ej.: a nivel de continentes, países, etc.
- Esta ley es válida sobre un tamaño de problema fijo, si varía el tamaño esta ley no se cumple.

Para ver si se cumple la ley de Amdahl debemos mirar la **tabla de Speedup por columnas**⁵. Dado que cada columna refiere a un problema de tamaño fijo, deberíamos ver que al aumentar las unidades de procesamiento el speedup aumenta hasta un cierto valor máximo debido a la parte no paralelizable.

9.5.2 Ley de Gustafson

A medida que aumentamos el tamaño del problema, la fracción “inherentemente secuencial” del programa disminuye en tamaño y es posible alcanzar mayor speedup.

En palabras sencillas, el overhead decrece en proporción del tamaño del problema. Cuando crece el tamaño del problema, la parte no paralelizable crece en comparación al problema.

Teniendo en cuenta un tamaño de problema infinito, se puede llegar a un speedup perfecto.

- Tiene en cuenta el tamaño del problema.
- Supone que mayor potencia de cómputo permitiría modelar problemas más profundamente.
- Supone que “sí se puede hacer algo más”.
- Esta ley es válida sobre un tamaño de problema variable, si se mantiene constante el tamaño está ley no se cumple.

Para ver si se cumple la ley de Gustafson debemos mirar la **tabla de Speedup por filas**⁶. Dado que cada fila refiere a un problema de tamaño variable para UP fijas, deberíamos ver que al aumentar el tamaño del problema el speedup aumenta hasta **P**.

9.5.3 Escalabilidad

Propiedad deseable de un sistema, red o proceso, que indica su **habilidad para reaccionar y adaptarse sin perder calidad**, manejar el crecimiento continuo de trabajo y estar preparado para hacerse más grande sin perder calidad en los servicios ofrecidos.

El análisis de escalabilidad nos da una idea teórica del comportamiento del sistema al incrementar su capacidad de cómputo.

Nos permite:

- Comparar el comportamiento de diferentes algoritmos.
- Predecir efectos en el rendimiento del sistema al cambiar alguno de sus parámetros.
- Optimizar la implementación paralela.

Un programa paralelo puede ser:

- **Fuertemente escalable:** mantiene la eficiencia constante sin incrementar el tamaño del problema.
¿Cómo analizo si una solución es fuertemente escalable? Para analizar este tipo de escalabilidad debemos mirar la **tabla de escalabilidad por columnas**, si los valores se mantienen mas o menos constantes, entonces es fuertemente escalable. Puede que sea fuertemente escalable a partir de un cierto valor, por lo tanto, se deben analizar todas las columnas.
- **Débilmente escalable:** mantiene la eficiencia constante al incrementar el tamaño del problema al mismo tiempo que se incrementa el numero de unidades de procesamiento.
¿Cómo analizo si una solución es débilmente escalable? Para analizar este tipo de escalabilidad debemos mirar la **tabla de escalabilidad por diagonales (generalmente la diagonal principal)**, si los valores se mantienen mas o menos constantes, entonces es débilmente escalable.
- **No escalable:** no se cumplen los casos anteriores.

⁵Generalmente en la primer columna, esto permite ver bien el efecto ya que el tamaño de problema es el más chico.

⁶Generalmente en la última fila, esto permite ver bien el efecto ya que permite crecer mucho el speedup.

9.5.4 Modelo de isoeficiencia

Las leyes de Amdahl y Gustafson se conocen también como modelos de speedup.

- **Modelo de carga fija (Amdahl):**

- Está asociado con **escalabilidad fuerte**, donde el tamaño del problema se mantiene constante.
- **El objetivo de este modelo es resolver un problema dado lo más rápido posible.** El mejor escenario es que el programa se ejecute P veces más rápido con P procesadores (speedup lineal).

- **Modelo de tiempo fijo (Gustafson):**

- Está asociado con la **escalabilidad débil**, donde **el tiempo de ejecución se mantiene fijo**, y lo que se busca es resolver problemas más grandes a medida que se agregan más procesadores.
- **El objetivo acá es mejorar la calidad o el detalle del resultado** (por ejemplo, más resolución, más precisión) **sin aumentar el tiempo de ejecución**. En este caso, el speedup ideal ocurre cuando logramos resolver un problema P veces más grande usando P procesadores, en el mismo tiempo que tomaría resolver el problema original de forma secuencial.

Definición (Sistemas Paralelos escalables): combinaciones de algoritmos y arquitecturas que cumplen la condición de mantener la eficiencia constante al incrementar simultáneamente el número de unidades de procesamiento y el tamaño del problema.

El **modelo de isoeficiencia** se usa para entender cuánto tiene que crecer el tamaño del problema para mantener constante la eficiencia de un sistema paralelo al aumentar el número de procesadores.

La **función de isoeficiencia** da información acerca de cómo debe crecer el tamaño del problema en función del número de unidades de procesamiento para poder mantener la eficiencia constante en Sistemas Paralelos escalables.

9.6 Métricas en sistemas con arquitecturas heterogéneas

Suponer $\mathbf{P}$ unidades de procesamiento con distinta potencia de cómputo $C_0, C_1, \dots, C_p$ y llamamos C_{base} a la potencia de cómputo de la UP más potente.

Entonces la eficiencia es:

$$E = \frac{C_{base}}{C_{promedio}} \cdot \frac{T_{secuencialBase}}{p \cdot T_p}$$

Despejando y normalizando la potencia base obtenemos:

$$S_{het} \leq C_{total}$$

La potencia de cómputo relativa expresada en función del tiempo de ejecución:

$$Pcr_i = \frac{T_{base}}{T_i}$$

Siendo T_i el tiempo que tarda una aplicación en ejecutarse en esa unidad de procesamiento, en particular T_{base} es el tiempo en la UP más potente.

En la práctica, si tenemos equipos multicore homogéneos, que resulta en una arquitectura heterogénea es más difícil de calcular.

9.7 Ejecución paramétrica

Si una aplicación debe ser ejecutada varias veces sobre una arquitectura se pueden seguir dos estrategias:

- **Ejecutar simultáneamente un mismo algoritmo secuencial con parámetros distintos para cada UP.**
- **Ejecutar consecutivamente una versión paralela (P veces), cada ejecución con parámetros distintos.**

Ambas estrategias, ante un speedup perfecto tardan el mismo tiempo. Entonces, podemos analizar cuando la ejecución paramétrica de forma paralela es conveniente:

- Speedup superlineal.
- Obtención de datos parciales: permite obtener las salidas para ciertos parámetros. La opción secuencial tiene los resultados al final.
- Aplicaciones de tiempo real. Permite tener los resultados antes, ideal para este tipo de aplicaciones.

10 Paradigmas de programación paralela

Los paradigmas de programación paralela son clases de algoritmos que resuelven distintos problemas, pero tienen la misma estructura.

En cada paradigma los patrones de comunicación son muy similares y es posible encuadrar distintos tipos de problemas paralelos en uno o mas de estos paradigmas.

10.1 Paralelismo de datos - SIMD/SPMD

El paralelismo de datos es el paradigma más común y el más simple: las tareas se mapean estática o semiestáticamente a unidades de procesamiento, y luego los procesos o hilos realizan operaciones similares o iguales sobre diferentes datos.

Se aplica a:

- Estructuras regulares y estáticas (arreglos, matrices, etc).
- Poca dependencia de datos o ninguna (Embarrassing Parallel Problem).
- Leve dependencia de datos sobre problemas de frontera (Ej: imágenes).
- Simulación.

Por ej.: Sumarle 1 a todos los elementos de un vector.

10.2 Divide y vencerás

Se realizan tres etapas:

- **Dividir:** se divide el problema en etapas de forma recursiva. La descomposición puede ser a no poder subdividir más (no siempre se puede, no hay recursos infinitos) o puede ser una descomposición intermedia.
- **Computar:** Se sigue una estructura tipo árbol, y los procesos o hilos realizan el cómputo en las hojas.
- **Combinar/Reducir:** Cada subproblema se soluciona independientemente y sus resultados se combinan o reducen para producir el resultado final: se combinan (a partir de N datos se generan N de salida) o reducen (a partir de N datos se genera uno solo).

En este paradigma la idea es crear procesos o hilos dinámicamente en función de los recursos. Se aplica en algoritmos por naturaleza recursivos:

- Ordenación.
- Búsquedas.
- Map-Reduce.

Por ej.: Buscar el valor mínimo en un vector.

10.3 Pipelines

Puede pensarse como una línea de ensamblaje/montaje donde en cada etapa se realiza una acción diferente sobre los datos. La aplicación se subdivide en subproblemas y cada subproblema se debe completar para comenzar el siguiente. El máximo paralelismo se alcanza cuando el pipeline se llena, aunque puede tener procesos ociosos.

Se aplica a:

- Paralelismo a nivel de instrucción (ILP).
- Procesamiento gráficos (Pipeline gráfico).
- Procesamiento de señales.

Por ej.: Pipelining del CPU.

10.4 Algoritmos sistólicos

Los algoritmos sistólicos son un caso particular del paradigma pipelines. Los procesos o hilos cooperan sincrónicamente con un conjunto regular de procesos o hilos vecinos más cercanos.

Existen dos etapas que se repiten hasta que en conjunto todos resuelven el problema:

- Todos los procesos o hilos comunican.
- Todos los procesos o hilos realizan cómputo.

Se aplica:

- Procesamiento de imágenes.
- Ecuaciones diferenciales.
- Simulaciones.

Por ej.: problema de los n-cuerpos.

10.5 Master-Worker

En este paradigma existen dos tipos de procesos o hilos:

- **Master:** Controla la ejecución del programa. Se encarga de descomponer el problema en tareas. Distribuye las tareas a los diferentes workers de forma estática o por demanda. Eventualmente, puede tener rol de Worker y asignarse tarea. Recolecta los resultados parciales para procesar el resultado final. Puede encargarse de crear varios o todos los workers.
- **Workers:** Realizan trabajo. Solicitan o reciben tareas del Master (Por demanda). Envían los resultados al Master. Eventualmente podrían generar más tareas (aplicaciones con datos generados dinámicamente). Pueden ser creados estática o dinámicamente por el Master.

Este paradigma se utiliza cuando el diseño está dominado por la necesidad de **balancear la carga dinámicamente**. **Es sensible al overhead por comunicación o sincronización.**

Se aplica a:

- Carga de trabajo asociadas a procesos o hilos que son variables e impredecibles (cargas dinámicas). Realizar una distribución de carga estática puede llevar a desbalances de carga que afectan el rendimiento.
- Grandes volúmenes de datos centralizados.

Por ej.: calcular el promedio de muchas notas dividiendo los datos entre varios workers, y luego el master reúne los resultados parciales.

10.6 Task pools

Task pools o Work pools es similar al paradigma Master-Worker, donde se prescinde del Master y se accede directamente a las tareas desde un repositorio común (*bag of tasks*).

Se deben considerar los accesos concurrentes al pool para evitar condiciones de carrera.

Por ej.: sumar varios números en paralelo usando pocos hilos.

10.7 Modelos Híbridos

En algunos casos, se puede aplicar más de un paradigma a un problema específico, lo que resulta en un modelo de algoritmo híbrido.

Un modelo híbrido puede estar compuesto por múltiples paradigmas aplicados jerárquicamente o múltiples paradigmas aplicados secuencialmente a diferentes fases de un algoritmo paralelo.

Por ej.: Los datos pueden fluir por un pipeline y dentro de cada nodo del pipeline puede haber paralelismo de datos.

11 Resumen de Programación CUDA

11.1 ¿Por qué usar GPUs?

Las GPUs tienen muchos núcleos simples (manycore), ideales para tareas altamente paralelizables. Esto dio origen al concepto **GPCGPU** (General-Purpose Computing on GPUs).

Características clave:

- Modelo SIMD: todos los hilos ejecutan el mismo código.
- Excelente rendimiento en paralelismo de datos.
- Memoria compartida y jerárquica.
- Potencia ideal para tareas intensivas en cómputo.

Básicamente sirve para:

- Cómputo paralelo masivo: ejecutar una misma instrucción sobre múltiples datos.
- Procesamiento matemáticas sobre vectores, matrices o tensores. Por ej.: multiplicación de matrices, reducción.
- Tareas con baja dependencia de datos.

Ejemplos particulares: renderizado de imágenes 3D, entrenamiento de redes neuronales, etc.

11.2 Arquitectura de una GPU Nvidia

Una GPU se compone de:

- **Sistema de procesamiento:** varios multiprocesadores (SMs) con núcleos FP32, FP64, INT32 y Tensor Cores (desde 2017).
- **Sistema de memoria:**
 - **On-chip:** registros, memoria compartida, cachés.
 - **Off-chip:** memoria global, de constantes, texturas, local.

11.3 Modelo de Programación CUDA

CUDA es una plataforma y extensión de C para programar GPUs Nvidia. Requiere tener en cuenta:

Copia explícita de memoria:

- Los datos se transfieren manualmente desde CPU (Host) a GPU (Device) y viceversa.
- Funciones: `cudaMemcpy()`, `cudaMalloc()`, `cudaFree()`.

Organización de los hilos:

- **Grid:** conjunto de bloques.
- **Bloque:** conjunto de hilos (threads).
- **Thread:** hilo de ejecución.

Dimensiones: se definen con `dim3`. Por ejemplo:

```
dim3 grid(2,2); dim3 block(4,4);
```

11.4 Kernel y ejecución

- Un **kernel** es la función que ejecutan los hilos.
- Se define con `__global__` y se invoca con `<<<grid, block>>>`.
- La ejecución es **asíncrona**; para sincronizar se usa `cudaDeviceSynchronize()`.

11.5 Identificación de hilos

- Se usan variables built-in como `threadIdx`, `blockIdx`, `blockDim`, `gridDim`.
- Ejemplo de ID único en 1D: `int id = blockIdx.x * blockDim.x + threadIdx.x;`

11.6 Limitaciones

- No hay E/S directa a disco desde GPU.
- No se permiten variables estáticas ni recursividad.
- Las funciones deben indicarse con `__host__`, `__device__` o ambos.

11.7 Tensor Cores

Unidades especiales para operaciones de matrices del tipo $D = AB + C$. Se usan en deep learning a través de bibliotecas como TensorFlow, PyTorch, CuBLAS, etc.

11.8 Métricas comunes

- **Speedup relativo:** $S = \frac{t_{CPU}}{t_{GPU}}$
- **Rendimiento efectivo:** GFLOPs reales medidos en ejecución.
- **Ancho de banda efectivo:** basado en datos transferidos y tiempo.

11.9 Depuración

Herramientas útiles:

- `nvprof` (profiling desde terminal)
- `cuda-gdb` (debugging)
- `nvvp` (Visual Profiler)

12 Jerarquía de memoria de GPUs

12.1 CUDA Capability

Una capability indica la generación de una GPU cuales son sus posibilidades y sus limitaciones. Una nueva capability agrega funcionalidad y tiene compatibilidad hacia atrás.

Una capability se representa por dos dígitos (R.V), el primero es la revisión (R) y el segundo es una versión (V). El mismo número de revisión indica la misma arquitectura.

Capability Arquitectura 1.0 G80 2.0, 2.1 Fermi 3.0, 3.1, 3.5 Kepler 5.0, 5.2, 5.3 Maxwell 8.0 Ampere

Distintas capabilities nos muestran distintos límites de dimensiones de los grids.

12.2 Características generales de la jerarquía de memoria Nvidia

- **On-chip:** Dentro de cada SM. registros, memoria compartida, cachés.
- **Off-chip:** Fuera de cada SM. memoria global, de constantes, texturas, local.

Cada nivel de memoria se diferencia en:

- Tamaño (GB a bits).
- Ancho de banda (TB a Mb).
- Velocidad de acceso (off-chip vs. on-chip).

El programador es quien decide de una variable:

- **El alcance:** que hilos acceden a una variable. Todos los hilos? solo uno? los hilos de un grid? etc.
- **El tiempo de vida:** duración de la variable en la ejecución del programa.
 - **La ejecución de un kernel:** la variable se crea durante la ejecución del kernel y se destruye cuando este finaliza.
 - **Toda la aplicación:** permanece accesible por todos los kernels en la aplicación.

12.3 Descripción de las distintas memorias de la jerarquía NVidia

Memoria	Ubicación	Cache	Acceso	Alcance	Tiempo de vida
Registro	On-Chip	No	Lectura y Escritura (Device)	Un hilo	Kernel
Local	Off-Chip	L1 y/o L2	Lectura y Escritura (Device)	Un hilo	Kernel
Compartida	On-Chip	No	Lectura y Escritura (Device)	Todos los hilos de un bloque	Kernel
Global	Off-Chip	L1 y/o L2	Lectura y Escritura (Device y Host)	Todos los hilos y el Host	Aplicación
Constantes	Off-Chip	Si	Lectura (Device) y Escritura (Host)	Todos los hilos y el Host	Aplicación
Texturas	Off-Chip	Si	Lectura (Device) y Escritura (Host)	Todos los hilos y el Host	Aplicación

12.3.1 Memoria global

- **Visibilidad:** R/W por CPU y GPU.
- **Velocidad:** Acceso lento (off-chip).
- **Variables:**
 - **Alcance:** Todos los hilos pueden accederla.
 - **Tiempo de vida:** Toda la aplicación.

Características extra:

- Los accesos a la memoria global se resuelven de a **medio warp** (16 hilos).
- Si los hilos acceden al mismo segmento de memoria, las transacciones pueden unificarse.
- La memoria global se divide en tres espacios de memoria independientes: la global ppd, la de texturas y la de constantes.

12.3.2 Memoria de constantes

- **Visibilidad:** R desde GPU. W desde CPU.
- **Velocidad:** Acceso lento (off-chip). Mejorado por cache on-chip.
- **Variables:**
 - **Alcance:** Todos los hilos de un grid y/o de una aplicación.
 - **Tiempo de vida:** Toda la aplicación.

Características extra:

- La memoria de constantes es parte de la memoria global.
- El tamaño de la memoria de constantes y su cache depende de la capability.

12.3.3 Memoria de texturas

- **Visibilidad:** R desde GPU. W desde CPU.
- **Velocidad:** Acceso lento (off-chip). Mejorado por cache on-chip.
- **Variables:**
 - **Alcance:** Todos los hilos de un grid y/o de una aplicación.
 - **Tiempo de vida:** Toda la aplicación.

Características extra:

- La memoria de texturas es parte de la memoria global, pero el costo de acceso no es el mismo que con el resto de la memoria global: la de texturas esta optimizada para aprovechar la localidad espacial de dos dimensiones (lecturas más rápidas de estructuras bi-dimensionales).

12.3.4 Memoria compartida

- **Visibilidad:** R/W solo desde GPU.
- **Velocidad:** Acceso rápido (on-chip).
- **Variables:**
 - **Alcance:** Todos los hilos de un bloque.
 - **Tiempo de vida:** Un kernel.

Características extra:

- Organizada en bancos, formado por palabras de 32 bits.
- Los accesos se resuelven de a medio warp (16 hilos).

12.3.5 Registros

- **Visibilidad:** R/W solo desde GPU.
- **Velocidad:** Acceso rápido (on-chip). Acceso de costo nulo y altamente paralelo.
- **Variables:**
 - **Alcance:** Hilos individuales (privacidad total).
 - **Tiempo de vida:** Un kernel.

Características extra:

- Se guardan valores escalares, no arreglos.
- Son limitados.

12.3.6 Memoria local

- **Visibilidad:** R/W solo desde GPU.
- **Velocidad:** Acceso lento (off-chip por estar en memoria global).
- **Variables:**
 - **Alcance:** Hilos individuales (privacidad total).
 - **Tiempo de vida:** Un kernel.

Características extra:

- Se almacenan **arreglos o escalares que no tienen lugar en los registros** (mayor demora que a los registros).

13 Optimizaciones CUDA

Para alcanzar un buen rendimiento sobre GPUs es necesario aprovechar las ventajas de la arquitectura subyacente. Para ello necesitamos conocer las características y factores limitantes en el rendimiento de las GPU.

13.1 Coalescence: Organización de los accesos

El **acceso a memoria global (DRAM)**⁷ es uno de los aspectos a considerar cuando se quiere ajustar el rendimiento de una aplicación en GPU.

Este tipo de optimización se basa en optimizar los accesos a posiciones consecutivas, el hardware combina todos estos accesos en un único acceso (coalescente). Tanto la lectura de los datos como la escritura, con previo procesamiento de los datos en memoria compartida, se busca realizar de forma coalescente.

Por ej.: acceder por columnas a una matriz almacenada por filas. Esto permite que los distintos hilos accedan a una misma fila (posiciones contiguas).

13.2 Prefetching: Técnicas de carga anticipada de datos

CUDA provee mecanismos para reducir la latencia de la memoria global. El modelo de multithreading por hardware permite que **mientras unos warps esperan por una operación de acceso a memoria otros ejecuten operaciones menos costosas**.

Para ello entre operaciones con accesos a memoria global existen instrucciones independientes, que son aquellas instrucciones que al momento de ejecutarse tienen los datos que necesita disponibles.

13.3 Divergence: Relacionado a la ejecución de los threads

El máximo paralelismo en un warp se alcanza si sus hilos no divergen: Una divergencia ocurre cuando los hilos de un warp ejecutan sentencias condicionales y no todos los hilos siguen el mismo camino. Dado que todos los hilos de un warp ejecutan la misma instrucción (SIMD - SIMT) se busca eliminar la divergencia.

Dependiendo del algoritmo se podría organizar la ejecución de manera que los hilos de un mismo warp sigan todos el mismo camino.

Por ej.: sumar los elementos de un vector.

13.4 Mezcla y granularidad: técnica de optimización en GPU

En GPU, no todas las instrucciones tienen el mismo costo. Las operaciones en punto flotante, accesos a memoria y *branch* son más costosas que instrucciones simples.

Mezcla de instrucciones: combinar instrucciones lentas y rápidas reduce el rendimiento, ya que los hilos de un *warp* deben esperar a los más lentos. Se busca minimizar esta mezcla y reorganizar el código para evitar cuellos de botella.

Granularidad: las GPU favorecen aplicaciones de **grano fino**, donde cada hilo realiza tareas pequeñas. Esto maximiza el paralelismo y mejora el uso de los recursos.

Técnica	Objetivo
Mezcla	Evitar combinar instrucciones con distinto costo para mejorar el rendimiento.
Granularidad	Usar grano fino: tareas pequeñas por hilo para aprovechar el paralelismo.

⁷Realizan lecturas simultáneas de una posición y sus vecinos más cercanos.

13.5 Occupancy: asignación de recursos en un SM

Cada SM (Streaming Multiprocessor) tiene recursos limitados: cantidad de bloques, hilos, registros y memoria compartida. Estos recursos deben repartirse entre los bloques activos.

Una buena *occupancy* busca maximizar la cantidad de hilos y bloques por SM, pero la configuración óptima depende de:

- La arquitectura (compute capability).
- El uso de registros por hilo.
- El uso de memoria compartida por bloque.
- El balance entre cómputo y acceso a memoria.

Consideraciones:

- Cada recurso puede limitar la cantidad de bloques por SM.
- La mejor configuración se determina de forma empírica.
- No existe una fórmula única; el programador debe analizar los recursos involucrados.

14 CUDA Streams

Para ocultar las latencias de copiar datos y resultados entre CPU y GPU, en ambos sentidos, se utilizan Streams: un stream es una cola de trabajo para el device (GPU), el host escolta el trabajo y sigue, cuando el device esta libre se planifica el trabajo de los streams.

- Las operaciones de un stream se ejecutan en FIFO.
- Las operaciones de distintos stream se ejecutan de forma desordenada.

Los llamados asíncronos deben converger en algún momento y debe existir alguna función que detenga la ejecución en el Host hasta que estos llamados se completen (Sincronización).

Los eventos son mecanismos para señalar si una operación ocurrió en un stream dado. por defecto el estado es “Ocurrió”.

15 MultiGPUs y modelos híbridos

En la actualidad existe una diversa cantidad de hardware muy potente. Las redes de comunicación permiten unir toda esta potencia de cómputo en Cluster/Multicluster formando un modelo híbrido.

Para realizar el cómputo correcto de un conjunto de datos, utilizando varias GPUs, se suele crear un hilo por cada GPU disponible. Cada hilo se encarga de gestionar todo lo relacionado con su propia GPU.